CADERNO DE ROTEIROS PRÁTICOS

Unidade Curricular: Tópicos Avançados em Informática

Ano 3 – 60h

Sumário

1 Roteiro 01 – Organização Profissional com Git e GitHub **4**

2 Roteiro 02 – Interação com APIs Web e Formatos de Dados **14**

3 Roteiro 03 – Flask Estruturado e Testes com Postman **17**

4 Roteiro 04 – Docker, Backup e Segurança Básica **20**

5 Roteiro 05 – Integração Frontend com API **23**

6 Roteiro 06 – Exploração Comparativa de Frameworks Web **26**

7 Roteiro 07 – Integração de IA no Desenvolvimento **29**

8 Roteiro 08 – Documentação Técnica com LaTeX no Overleaf **32**

9 Roteiro 09 – Projeto Integrador Final **35**

1 Roteiro 01 – Organização Profissional com Git e GitHub

Objetivos

- Compreender o conceito de controle de versão;
- Utilizar comandos básicos do Git;
- Criar e manter repositórios no GitHub;
- Organizar profissionalmente um projeto de software.

Ferramentas Necessárias

Git instalado, VSCode (ou IDE similar), conta no GitHub, terminal Linux.

Introdução

No desenvolvimento profissional de software, manter histórico organizado de alterações é fundamental. O controle de versão permite registrar modificações, recuperar versões anteriores, trabalhar em equipe e manter a rastreabilidade das mudanças realizadas no código.

Nesta aula, aprenderemos a utilizar o Git como ferramenta de controle de versão e o GitHub como repositório remoto para armazenamento e colaboração.

Fundamentação Teórica

Controle de versão é o processo de registrar mudanças feitas em arquivos ao longo do tempo, permitindo recuperar versões específicas posteriormente.

Repositório local: é a pasta do projeto na máquina do desenvolvedor, onde o Git controla as alterações.

Repositório remoto: é a versão armazenada em um servidor (ex.: GitHub), permitindo backup e colaboração.

Commit: registro de alterações realizadas no projeto.

Branch: linha alternativa de desenvolvimento utilizada para implementar funcionalidades ou testes sem afetar o código principal.

Merge: integração de alterações realizadas em uma branch ao ramo principal.

O Git é um sistema de controle de versão distribuído, amplamente utilizado na indústria de software.

Roteiro Prático

Atividade prática 1 — Configuração Inicial do Git

Nesta etapa, iniciaremos a configuração do ambiente Git, preparando o sistema para registrar corretamente as alterações realizadas no projeto.

- Verificar instalação do Git:

```
git --version
```

- Configurar nome e e-mail (usar o mesmo do GitHub):

```
git config --global user.name "Seu Nome"
git config --global user.email "seuemail@email.com"
```

- Confirmar configurações:

```
git config --list
```

Agora criaremos nosso primeiro projeto versionado, organizando corretamente a estrutura inicial do repositório local.

Atividade prática 2 — Criando o Projeto Local

Agora criaremos nosso primeiro projeto versionado, organizando corretamente a estrutura inicial do repositório local.

- Criar diretório do projeto:

```
mkdir projeto-git
cd projeto-git
```

- Inicializar repositório:

```
git init
```

- Criar arquivo README:

```
touch README.md
```

- Escrever breve descrição do projeto no README.

Após criar os arquivos iniciais, registraremos oficialmente o primeiro estado do projeto no histórico de versões.

Micro-Commit 1 — Primeiro Registro

```
git add README.md
git commit -m "Adiciona README inicial do projeto"
```

Agora exploraremos o uso de branches, prática essencial para desenvolvimento profissional e organização de funcionalidades.

Atividade prática 3 — Trabalhando com Branch

- Criar nova branch:

  ```
  git checkout -b feature-inicial
  ```

- Criar novo arquivo da aplicação:

  ```
  touch app.py
  ```

- Inserir comentário inicial no arquivo:

  ```
  # Aplicação inicial
  ```

Registraremos agora essa nova funcionalidade no histórico da branch criada.

Micro-Commit 2 — Nova Funcionalidade

```
git add app.py
git commit -m "Adiciona arquivo inicial da aplicação"
```

Em seguida, integraremos a nova funcionalidade à branch principal do projeto.

Atividade prática 4 — Merge com Main

```
git checkout main
git merge feature-inicial
```

Por fim, enviaremos o projeto para o GitHub, tornando-o disponível remotamente e criando um backup online.

Atividade prática 5 — Enviando para o GitHub

- Criar repositório no GitHub (sem README).
- Copiar a URL do repositório.
- Executar:

```
git remote add origin https://github.com/usuario/projeto-git.git
git branch -M main
git push -u origin main
```

- Verificar se os arquivos foram publicados corretamente.

Após o envio, verificar no navegador se o repositório foi publicado corretamente.

Boas Práticas de Commit

- Utilizar mensagens claras e objetivas;
- Evitar mensagens genéricas como "update"ou "teste";
- Realizar commits pequenos e frequentes;
- Manter organização no histórico.

Entrega

Cada estudante deverá:

- Criar repositório público no GitHub;
- Realizar no mínimo 3 commits organizados;
- Criar e integrar ao menos uma branch;

- Inserir descrição adequada no README;
- Enviar o link do repositório.

O histórico de commits será avaliado quanto à organização e clareza das mensagens.

Agora simularemos uma situação comum em equipes: conflito de versão.

Agora iremos simular um fluxo profissional utilizando Pull Request diretamente no GitHub.

Atividade prática 6 — Criando um Pull Request

- Criar nova branch:

```
git checkout -b melhoria-readme
```

- Alterar o README.md adicionando uma nova descrição.
- Realizar commit:

```
git add README.md
git commit -m "Melhora descrição do README"
```

- Enviar a branch para o GitHub:

```
git push origin melhoria-readme
```

- No GitHub, clicar em "Compare and Pull Request".
- Criar Pull Request.
- Aprovar e realizar o merge via interface web.

Atividade prática 7 — Resolvendo Conflito Simples

- Na branch main, alterar novamente a mesma linha do README.
- Commitar alteração:

```
git add README.md
git commit -m "Atualiza descrição principal"
```

- Criar nova branch a partir de uma versão anterior (ou alterar a mesma linha em outra branch).
- Tentar realizar merge:

```
git merge nome-da-branch
```

- Observar as marcações de conflito no arquivo:

```
<<<<<<< HEAD
=======
```

```
>>>>>>> branch
```

- Editar manualmente o arquivo escolhendo a versão correta.
- Finalizar com:

```
git add README.md
git commit -m "Resolve conflito no README"
```

Merge vs Rebase (Conceito)

Merge: mantém o histórico completo das branches, criando um commit de integração.

Rebase: reorganiza o histórico, reaplicando commits sobre outra base.

Nesta disciplina utilizaremos principalmente o **merge**, por ser mais seguro para iniciantes.

Nesta atividade iremos simular um ambiente real de desenvolvimento colaborativo, onde vários desenvolvedores trabalham simultaneamente no mesmo projeto utilizando branches individuais e Pull Requests.

Atividade prática — Colaboração em Repositório Compartilhado

Repositório oficial da atividade:
`https://github.com/chameoandre/operacoes-git-2.git`

Passo 1 — Acessar diretório de trabalho

Antes de clonar o repositório, sair de qualquer outro projeto Git:

```
cd ~
mkdir -p projetos-ifsc
cd projetos-ifsc
```

Passo 2 — Clonar o repositório oficial

```
git clone https://github.com/chameoandre/operacoes-git-2.git
cd operacoes-git-2
```

Passo 3 — Criar branch individual

Cada estudante deve criar sua própria branch seguindo o padrão:

```
git checkout -b aluno-seu-nome
```

Exemplo:

```
git checkout -b aluno-joao-silva
```

Passo 4 — Editar o arquivo de instruções

- Inserir seu nome abaixo da lista #Presentes;
- Remover um dos nomes existentes;

- Salvar o arquivo.

Passo 5 — Registrar alteração

```
git add README.md
git commit -m "Adiciona nome João Silva e remove Ignacio"
```

Passo 6 — Enviar branch para o GitHub

```
git push origin aluno-seu-nome
```

Passo 7 — Criar Pull Request

- Acessar o repositório no GitHub;
- Clicar em "Compare & Pull Request";
- Criar Pull Request para a branch `main`;
- Aguardar revisão e possível resolução de conflito.

Observação Importante

Como vários estudantes estarão modificando o mesmo arquivo, poderão ocorrer conflitos durante o merge.

Caso isso aconteça:

- Atualizar sua branch com `git pull origin main`;
- Resolver manualmente os conflitos;
- Realizar novo commit;
- Atualizar o Pull Request.

Conflitos fazem parte do processo profissional de desenvolvimento e devem ser tratados com organização e atenção.